1.State Preservation and Session Management in HTTP Applications

HTTP is a stateless protocol in which each request-response cycle is executed independently, with no built-in memory of past interactions. To preserve application state across multiple cycles, particularly for user authentication, modern web applications primarily rely on session identifiers or cryptographic tokens.
In a traditional session-based architecture, when a user authenticates successfully, the server creates a unique session record in its data store and transmits a corresponding session ID back to the client, typically encapsulated within an HTTP Set-Cookie header. The client's browser automatically appends this cookie to all subsequent requests, allowing the server to look up the active session and verify user identity. Alternatively, token-based authentication models, such as JSON Web Tokens (JWT), bypass server-side session storage by encoding user identity and permissions directly into a cryptographically signed payload. This token is sent to the client and generally stored in local Storage or secure, HTTP-only cookies, then attached to later requests via the HTTP Authorization: Bearer header. By validating the digital signature of incoming tokens or checking active session databases, applications seamlessly maintain a continuous, stateful experience over an otherwise disjointed protocol.

2. Django Database Migrations to MariaDB

Transitioning a Django application from a local development environment to a server-based relational database like MariaDB requires configuring proper backend drivers and executing sequential migration commands. The process begins by installing a compatible database adapter, such as the MySQL client, in the application environment to bridge Django and MariaDB. Next, the project's settings.py file must be updated within the DATABASES dictionary, specifying django.db.backends. MySQL as the engine alongside the server's host address, port, database name, and authenticated user credentials. Once network connectivity to the remote MariaDB instance is established and an empty target database is initialized, developers execute python manage.py makemigrations locally to package any outstanding schema changes into discrete Python files. Finally, the python manage.py migrate command is run against the production settings. This command reads the migration history table in MariaDB, determines which updates have not yet been executed, and safely translates the Django Python code into native SQL DDL (Data Definition Language) queries to construct tables, indexes, and constraints on the database server.